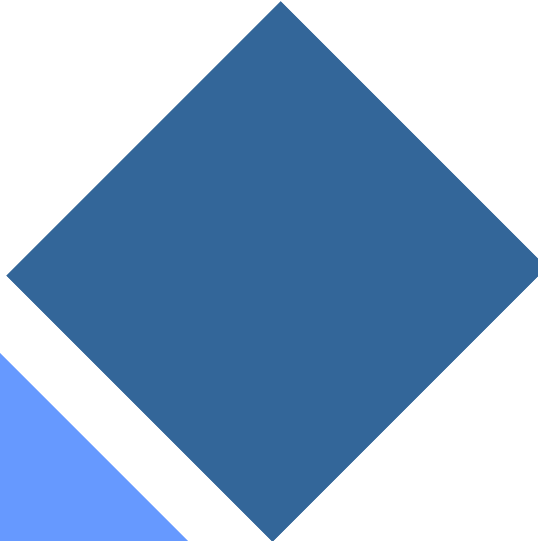


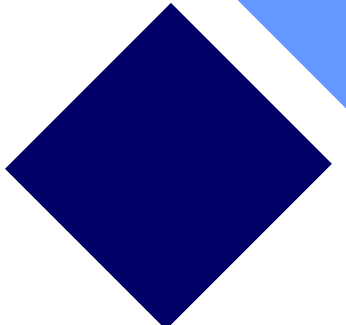
SP4: Limpieza con Spark SQL

Isabel Pelaya Galindo Ibáñez
Esther Ibáñez Mingorance
José Luis López García
Juan Rafael Madolell Usero





Indice



- 1. ¿QUÉ ES ETL?**
- 2. Calidad del dato**
- 3. Transformaciones en Spark**
- 4. No for**
- 5. Práctica**
- 6. Problemas**

1. ¿QUÉ ES ETL?

¿Qué es?: Es el proceso fundamental en la gestión de datos.

- **Extract:** se obtienen los datos desde diferentes fuentes (bases de datos, APIs, archivos CSV, etc.)
- **Transform:** se limpian, corrigen y adaptan los datos para su uso
- **Load:** se cargan los datos ya procesados en un sistema destino

Este proceso permite convertir datos brutos en información útil y fiable

2. CALIDAD DEL DATO

Un dato limpio debe ser:

- **Completo:** sin valores nulos innecesarios
- **Consistente:** mismo formato en todo el dataset
- **Correcto:** tipos de datos adecuados
- **Único:** sin duplicados

La calidad de los resultados de un sistema depende directamente de la calidad de los datos de entrada.

“Garbage in, garbage out”

ESTRATEGIAS DE CALIDAD DE DATOS

Manejo de Nulos:

- **df.dropna():** Elimina la fila. Útil si el dato es vital (ej. un ID).
- **df.fillna():** Imputación. Rellenar con la media, mediana o un valor por defecto.

Duplicados: *df.dropDuplicates()* es vital, pero recuerda que es una operación de tipo Shuffle.

Funciones de Ventana (Window Functions): Permiten hacer cálculos complejos sin colapsar las filas.

3. TRANSFORMACIONES EN SPARK

Las transformaciones se realizan principalmente sobre DataFrames.

- **Renombrado:** Lo usamos para cambiar el nombre de las columnas
Ejemplo: `withColumnRenamed("nombre_viejo", "nombre_nuevo")`.
- **Casting:** Es el proceso de cambiar el tipo de dato (ej. de String a Integer o Date).
Ejemplo: `df.withColumn("precio", col("precio").cast("double"))`
- **Lógica Condicional:** Spark te permite trabajar de dos formas
 - Programática (`withColumn`): Ideal para pipelines automatizados
 - Declarativa (Spark SQL): Uso de SQL puro

TRANSFORMACIONES POR RENDIMIENTO

No todas las transformaciones cuestan lo mismo a nivel de rendimiento.

- **Narrow transformations:** Cada nodo trabaja con sus propios datos, sin necesidad de comunicarse con otros.
- **Shuffle transformations:** Los datos tienen que mezclarse entre nodos para poder realizar la operación.

Conclusión: Las transformaciones narrow no requieren mover datos entre nodos, mientras que las shuffle sí, lo que las convierte en operaciones mucho más costosas en Spark.

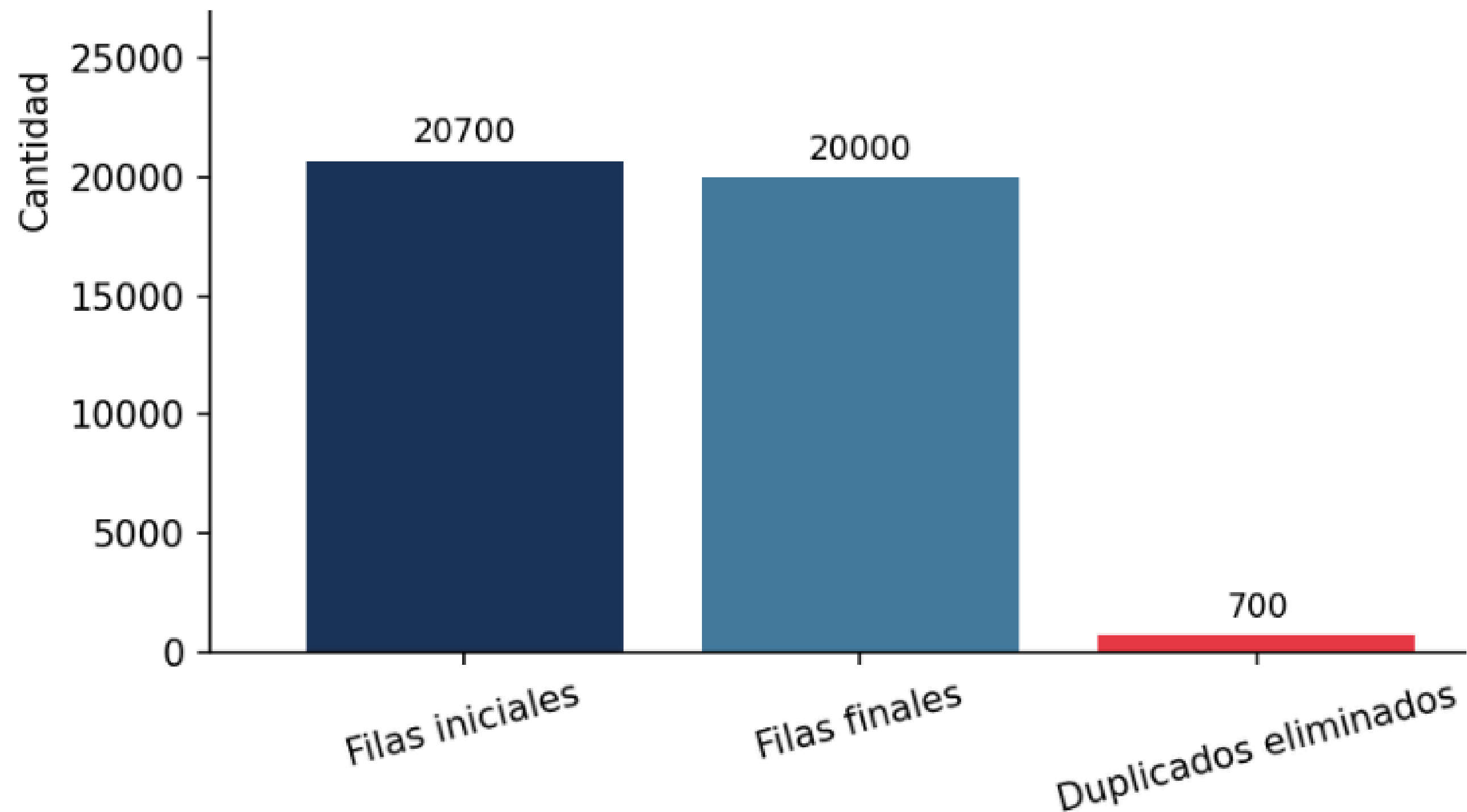
4. NO FOR

En programación tradicional usamos for para recorrer listas, pero en Spark es una **mala práctica**.

- **Computación Distribuida:** Spark reparte los datos en muchos nodos (máquinas). Un for es **secuencial**, obliga al driver a procesar fila por fila.
- **Funciones Vectorizadas:** Spark está optimizado para aplicar transformaciones a toda una columna a la vez en todos los nodos simultáneamente.

5. PRÁCTICA

Limpieza de los datos



5. PRÁCTICA

Limpieza de los datos

tipo_problema	columna	registros_corr	solucion	valor_usado
Fila duplicada	(todas)	700	Eliminación	—
Cadena vacía / espacios	gender	60	Convertido a NaN	(ver imputación)
NaN en columna numérica	study_hours_per_day	190	Media	5.2521
NaN en columna numérica	sleep_hours	150	Media	6.5188
NaN en columna numérica	coffee_intake_mg	130	Media	249.5378
NaN en columna numérica	exercise_minutes	110	Media	59.6755
NaN en columna numérica	attendance_percentage	75	Media	69.953
NaN en columna numérica	final_grade	60	Media	70.2684
NaN en columna numérica	productivity_score	35	Media	50.1757
NaN en columna categórica	gender	140	Moda	Female
TOTAL		1650		

5. PRÁCTICA

Creación de la columna nueva con withcolumn

Cargando: student_productivity_distraction_CLEAN.csv
Filas cargadas: 20,000

Aplicando withColumn → perfil_gamer ...

Criterios de clasificación

gaming_hours == 0 → No Juega

0 < h ≤ 2 (o ≤1.5 si age≤20) → Novato

2 < h ≤ 4 (o ≤3.5 si age≤20) → Gamer

h > 4 (o > 3.5 si age≤20) → Experto Gamer

Modificador gender='Other' → Novato desde >0h

Distribución de perfil_gamer:

perfil_gamer	count
Experto Gamer	7117
Gamer	6681
Novato	6185
No Juega	17

Distribución cruzada: género × perfil_gamer

gender	No Juega	Novato	Gamer	Experto Gamer
Female	10	2981	3281	3439
Male	7	2968	3136	3408
Other	NULL	236	264	270

5. PRÁCTICA

Tiempo que tarda en crear la columna

Distribución cruzada: género × perfil_gamer

gender	No Juega	Novato	Gamer	Experto Gamer
Female	10	2981	3281	3439
Male	7	2968	3136	3408
Other	NULL	236	264	270

Media de gaming_hours por perfil:

perfil_gamer	media_gaming_h	media_edad	total
Experto Gamer	4.92	22.7	7117
Gamer	2.84	23.0	6681
Novato	0.94	23.4	6185
No Juega	0.0	22.8	17

```
=====
✓ Filas procesadas      : 20,000
✓ Tiempo withColumn+eval: 1213.47 ms (1.213 s)
=====
```

```
Guardando resultado en: student_productivity_distraction_GAMER.csv
✓ Guardado correctamente
```

5. PRÁCTICA

Dataset resultante

assignments_completed	attendance_percentage	stress_level	focus_score	final_grade	productivity_score	perfil_gamer
2	57.21	10	57	81.87	33.78	Experto Gamer
10	91.27	10	49	60.9	48.99	Gamer
8	63.14	2	38	86.22	36.6	Experto Gamer
18	40.51	6	50	71.77	19.87	Gamer
7	45.53	6	41	90.13	52.9	Experto Gamer
3	47.58	10	70	59.48	47.31	Novato
15	43.5	8	35	62.71	41.23	Experto Gamer
17	75.22	6	59	52.22	53.81	Experto Gamer
11	44.79	3	39	76.15	25.99	Gamer
6	79.15	10	73	88.53	73.18	Gamer
2	54.99	10	82	76.52	55.34	Novato
13	86.6	10	92	97.36	77.35	Novato
3	77.02	3	86	53.59	78.35	Experto Gamer
7	93.99	4	78	97.9	76.03	Novato
6	67.85	8	96	49.08	47.86	Experto Gamer
10	63.49	5	97	58.24	67.31	Gamer

6. PROBLEMAS

Problema:

Pandas usa `isnull()` que solo detecta NaN, None y NaT, pero no cadenas vacías ni espacios en blanco.

Solución:

Se desarrolla una función para que todo lo vacío o espacio en blanco se convierta null.

6. PROBLEMAS

Problema:

El DataSet no presentaba datos erróneos.

Solución:

Hemos tenido que añadir datos erróneos al dataset para poder comprobar que la limpieza se hacía correctamente

**Muchas
gracias**

